Physics LM Canon Layers Presentation

Assembled Agent Prompts

10 tasks • 18 edges • Exported March 11, 2026

> Each card below shows the **full assembled prompt** that Astro's dispatch engine sends to the agent for each task. This mirrors `ContextAssembler.buildExecutionPrompt()` — the sections are assembled at dispatch time from the project vision document, dependency graph, completed task outputs, and the task description. Sections not available at export time (skill instructions, runtime knowledge) are omitted.

Vision | Context | Dependencies | Task (prompt) | Instructions

Tasks

- #1 Paper Analysis & Narrative Outline
- #2 Slides 1-11: Title, Motivation, Background, Synthetic Playground
- #3 Slides 12-22: Canon Layers + Part 4.1 Results
- #4 Slides 23-33: Part 4.2 Scale Results, Error Analysis, Conclusion
- #5 Blog Post Generation
- #6 Poster Generation
- #7 Figure Extraction and Preparation
- #8 PDF Assembly and Quality Check
- #9 Content Review and Narrative Polish
- #10 All Deliverables Complete

#1 Paper Analysis & Narrative Outline

PROJECT VISION

▶ 47,429 chars — click to expand

TASK CONTEXT

```
18 tasks depend on this one.
```

CURRENT TASK

```
**Paper Analysis & Narrative Outline**

Deep analysis of both Part 4.1 (paper) and Part 4.2 (code+results) to produce a
comprehensive narrative outline.

## What to do
1. **Read the full LaTeX source** at `paper_source/canon-5arxiv2-ob.tex` (4317 lines).
Parse all 12 Results, extract key equations (Canon layer formula, Canon-res formula),
figure references, and the narrative arc.
2. **Read Part 4.2 materials**: `PhysicsLM4/canon_llama_results/README.md` and
`PhysicsLM4/canon_linear_results/README.md` for detailed findings. Read the performance
tables from the PNG/PDF images.
3. **Read the Canon layer implementation** in `PhysicsLM4/huggingface/canon_helper.py` —
understand the `ShortConvolution` class, `create_canon()`, and `apply_canon()` functions.
4. **Read the GLA/Mamba2/GDN model code** in
`PhysicsLM4/lingua_modified/apps/gla/transformer.py` to understand how Canon integrates
with linear models.
5. **Build a figure inventory**: Catalog all usable figures from `paper_source/canon-figs5-
arxiv/plots.pdf` (pages 3-8), experiment subdirectories, and
`PhysicsLM4/canon_llama_results/` + `PhysicsLM4/canon_linear_results/` images. For each,
note: path, what it shows, which slide it maps to.
6. **Identify SVGs to generate**: Architecture diagrams (Transformer block with Canon-ABCD
points), comparison diagrams (before/after Canon), error accumulation illustration,
timeline of architecture families.

## Output
Write `outline/narrative-outline.md` containing:
- Paper comprehension (800+ words covering both 4.1 and 4.2)
- Story arc with 9 acts (Hook → Background → Synthetic Playground → Canon Key Insight → 4.1
Transformer Results → 4.1 Linear Results → 4.2 Scale Results → Error Analysis → Conclusion)
- Complete figure inventory with slide assignments
- Slide-by-slide plan for ~33 slides, each with: title, layout type (1-5), key points (3-5
bullets), figure plan ([original: path] or [SVG: description]), source attribution (4.1 or
4.2), transition to next slide
- Both light and dark theme will be generated for each slide

## Key Narrative Decisions
- Audience: experts in mechanistic interpretability or LM training
- Hook: Start with the noise/unreliability problem in architecture comparison
- Key insight slide: Canon = horizontal residual links, like the musical canon
- Part 4.2 must get equal depth: dedicate ~8 slides to scaling results + error analysis
- Always attribute results to source (4.1 paper or 4.2 code/results)
```

```
- Include in-depth error analysis for linear models (memory dynamics, retrieval failures,
2-hop reasoning)

## Working Directory
`/Users/zhuoran_cisco/Documents/astro-outputs/presentation/physcics-llm-2/`

## Done Criteria
- `outline/narrative-outline.md` exists with complete slide-by-slide plan
- Figure inventory covers ≥20 original figures + ≥8 new SVG plans
- Every slide has layout type, figure plan, and source attribution
```

INSTRUCTIONS

```
Execute the task described above. Focus on completing the task thoroughly and correctly.
- Use appropriate tools to accomplish the task
- Report progress and any issues encountered
- Verify your work when possible
- Be thorough but efficient
- When you make code changes, commit them with a descriptive commit message before finishing.
Do not leave uncommitted changes.
```

#2 Slides 1-11: Title, Motivation, Background, Synthetic Playground

PROJECT VISION

▶ 47,429 chars — click to expand

TASK CONTEXT

```
This task is at depth 2 in the plan graph. It has 3 upstream dependencies. 3 tasks depend on
this one. 4 other tasks are running in parallel.
```

CURRENT TASK

```
**Slides 1-11: Title, Motivation, Background, Synthetic Playground**

Generate HTML slides 1-11 (both light and dark themes) covering the opening acts of the
presentation.

## Slides to Generate
All slides are self-contained HTML files at 1280×720px. Generate BOTH
`output/slides/light/slide_XX.html` and `output/slides/dark/slide_XX.html` for each. Follow
the SKILL.md layout types, color palettes, and typography exactly.

### Slide 1 — Title Slide (Layout 1)
- Title: 'Physics of Language Models: Part 4, Architecture Design and the Magic of Canon
Layers'
- Author: Zeyuan Allen-Zhu (FAIR at Meta)
- Subtitle: 'Covering Part 4.1 (NeurIPS 2025) and Part 4.2 (Code Release)'

### Slide 2 — Section Divider (Layout 2): 'The Challenge'
- Subtitle: 'Why comparing architectures is harder than you think'

### Slide 3 — Real-life Pretraining is Noisy (Layout 4: Left-Right Split)
- Left: 3 challenges (pretraining loss unreliable, noise below emergence, grokking/data
quality)
- Right: Use figure from `paper_source/canon-figs5-arxiv/plots.pdf` page 8
(fig:illustrate1) — embed as <img> referencing the PDF extraction or recreate key elements
as SVG showing noisy benchmarks vs clean synthetic
- Source: [4.1]

### Slide 4 — Architecture Landscape (Layout 3: Full Bullet)
- Overview of 4 architecture families: Transformer (Llama/RoPE/NoPE), Linear Attention
(GLA), SSM (Mamba2), Gated DeltaNet (GDN)
- Brief characterization of each + parameter standardization approach
- Generate SVG: timeline/taxonomy of architecture families with color-coded nodes
- Source: [4.1]

### Slide 5 — Academic Scale Limitations (Layout 4: Left-Right Split)
- Left: Key stats — 1.3B params, 100B tokens → 2-4% swings from random seeds, 2-hop
reasoning fails
- Right: Use figure from plots.pdf page 8 showing benchmark variability across seeds
- Source: [4.1]

### Slide 6 — Section Divider (Layout 2): 'Synthetic Pretraining Playground'
```

- Section number: 02
- Subtitle: 'Atomic decomposition of intelligence'

Slide 7 — Design Criteria (Layout 4: Left-Right Split)
- Left: 4 criteria (challenge depth, test mental reasoning, avoid length gen emphasis, real-world relevance)
- Right: Use figure from plots.pdf page 3 (design criteria figure) or recreate as SVG
- Source: [4.1]

Slide 8 — Five Synthetic Tasks Overview (Layout 4: Left-Right Split)
- Left: List of 5 tasks with one-line descriptions
- Right: Use figure from plots.pdf page 4/9 (five tasks illustration) or recreate as SVG showing the 5 tasks with icons
- Source: [4.1]

Slide 9 — Task Details: Depo + Brevo (Layout 3: Full Bullet)
- Depo: k-hop traversal over directed permutations, measures reasoning depth
- Brevo: Recursive DAG traversal, measures reasoning breadth
- Include task format examples: `<bos> x1 y1 x2 y2 ... <query_k> q a`
- Generate SVG: Simple directed graph showing k-hop path
- Source: [4.1]

Slide 10 — Task Details: Capo + Mano + Lano (Layout 3: Full Bullet)
- Capo: Knowledge capacity (bits per parameter), synthetic biographies
- Mano: Knowledge manipulation, modular arithmetic expressions
- Lano: Hierarchical language structure, CFG parsing
- Generate SVG: Expression tree for Mano example
- Source: [4.1]

Slide 11 — Initial Comparison Results (Layout 5: Three-Zone)
- Top-left: Key findings — GLA weakest, Mamba2 excels at knowledge, GDN strong reasoning, RoPE best depth
- Top-right: Use heatmap figures from `paper_source/canon-figs5-arxiv/perm_4/` or create summary SVG bar chart
- Bottom: 'This comparison is incomplete — Mamba2/GDN have conv1d but GLA/Transformer don't → unfair!'
- Source: [4.1]

File Paths
- Light slides: `output/slides/light/slide_01.html` through `slide_11.html`
- Dark slides: `output/slides/dark/slide_01.html` through `slide_11.html`
- Generated SVGs: inline within HTML
- Paper figures: reference via relative path from `paper_source/canon-figs5-arxiv/`

Technical Requirements
- Include KaTeX CDN in any slide with math
- All original paper figures: copy relevant PNGs/extract PDF pages to `output/figures/` for embedding
- Use `<img>` for raster figures, inline `<svg>` for generated diagrams
- Follow SKILL.md color palette exactly (light: bg #FAF8F5, text #3A3A3A; dark: bg #2A2A2E, text #E8E4E0)
- Max 2-3 accent colors per slide

Done Criteria
- 22 HTML files exist (11 light + 11 dark)
- Each is self-contained, renders at 1280×720

```
- ≥7 slides have figures (original or SVG)
- All SVGs use palette colors with no text overlap
```

INSTRUCTIONS

```
Execute the task described above. Focus on completing the task thoroughly and correctly.
- Use appropriate tools to accomplish the task
- Report progress and any issues encountered
- Verify your work when possible
- Be thorough but efficient
- When you make code changes, commit them with a descriptive commit message before finishing.
Do not leave uncommitted changes.
```

#3 Slides 12-22: Canon Layers + Part 4.1 Results

PROJECT VISION

▶ 47,429 chars — click to expand

TASK CONTEXT

```
This task is at depth 2 in the plan graph. It has 3 upstream dependencies. 3 tasks depend on
this one. 4 other tasks are running in parallel.
```

CURRENT TASK

```
**Slides 12-22: Canon Layers + Part 4.1 Results**

Generate HTML slides 12-22 (both light and dark themes) covering Canon layers and all Part
4.1 synthetic results.

## Slides to Generate

### Slide 12 — Section Divider (Layout 2): 'Canon Layers'
- Section number: 03
- Subtitle: 'Enhancing horizontal information flow'

### Slide 13 — Why Canon? The Associative Recall Problem (Layout 4: Left-Right Split)
- Left: Explain the folklore — associative recall `[A] [B] ... [A] [?]` needs 2 attention
layers because causal masking prevents attending backward. 'Shooting a bird with a cannon'
— using global attention just to pass info to neighbors.
- Right: Use figure from plots.pdf page 5 showing the two-layer attention mechanism for
recall. Show the problem and solution.
- Include equation context: first attention stores A→B, second retrieves B via key=A
- Source: [4.1]

### Slide 14 — Canon Layer Definition (Layout 4: Left-Right Split)
- Left: The Canon formula: $h'_t = w_0 \odot h_t + w_1 \odot h_{t-1} + w_2 \odot h_{t-2} +
w_3 \odot h_{t-3}$
- Musical analogy: Pachelbel's Canon — overlapping melodies at fixed temporal delays
- Implementation: 1-d causal convolution, kernel size 4, with residual: $h' = h +
\text{conv1d}(h)$
- Right: Use Canon layer illustration from plots.pdf page 6 — the horizontal arrows showing
delayed repetition
- Source: [4.1]

### Slide 15 — Canon-ABCD Integration Points (Layout 3: Full Bullet + SVG)
- Canon-A: Before attention (m=d), after RMSnorm
- Canon-B: Inside attention, after Q/K/V projections (m=3d)
- Canon-C: Before MLP (m=d), after RMSnorm
- Canon-D: Inside MLP (m=4d or 16d/3 for gated), before activation
- Generate SVG: Transformer block diagram with 4 Canon insertion points highlighted in
different colors
- Highlight: <0.45% parameter increase for GPT2-small, 12-20% runtime overhead on H100
- Source: [4.1]

### Slide 16 — Code: Canon Implementation (Layout 3: Full Bullet)
```

- Show simplified code from `canon_helper.py`: the `create_canon()` and `apply_canon()` functions
- Key insight: Just a few lines of code change — `ShortConvolution` wrapping `nn.Conv1d` with causal padding
- Residual connection: `if canon._zeyuan_residual: return hidden_states + hidden_states2`
- Use code font styling (Fira Code)
- Source: [4.1 + 4.2 code]

Slide 17 — Section Divider (Layout 2): 'Results: Transformer + Canon'
- Section number: 04
- Subtitle: 'Part 4.1 — Synthetic pretraining results'

Slide 18 — Result 2: RoPE + Canon Dramatically Improves Reasoning (Layout 5: Three-Zone)
- Top-left: Key findings — reasoning depth improves 200-400%, breadth +30%, knowledge manipulation +30%
- Top-right: Use heatmap comparison figures from `perm/` directory showing RoPE original vs RoPE+Canon-ABCD
- Bottom: 'Canon enables hierarchical learning dynamics with minimal computational overhead'
- Source: [4.1, Result 2]

Slide 19 — Result 3: NoPE + Canon = Strong Performer (Layout 5: Three-Zone)
- Top-left: NoPE (no positional embeddings) was weak → with Canon, matches or surpasses RoPE! RoPE(¼)+Canon is best — great for length generalization.
- Top-right: Figure comparison of NoPE original vs NoPE+Canon columns from fig:trans-canon
- Bottom: 'Partial RoPE (¼ dimensions) + Canon outperforms both → reduces RoPE dependency for better length generalization'
- Source: [4.1, Result 3]

Slide 20 — Results 4-5: Ablations + MLP/MoE (Layout 3: Full Bullet)
- Canon-A/B/C/D contribute cumulatively; each independently useful
- Residual Canon improves training efficiency
- Canon recovers knowledge capacity lost in gated MLP and MoE architectures
- Generate SVG: Bar chart showing incremental contribution of Canon-A, AB, ABC, ABCD
- Source: [4.1, Results 4-5]

Slide 21 — Section Divider (Layout 2): 'Results: Linear Models + Canon'
- Section number: 05
- Subtitle: 'Part 4.1 — GLA, Mamba2, GDN meet Canon'

Slide 22 — Results 6-8: Universal Boost for Linear Models (Layout 5: Three-Zone)
- Top-left: GLA+Canon: reasoning depth 1→4 hop, doubles breadth. Mamba2's conv1d ≈ Canon-B. GDN benefits least (already has Canon-like behavior).
- Top-right: Use comparison heatmaps from experiment directories or create SVG summary
- Bottom: 'Canon-ACD alone matches internal conv1d → horizontal mixing useful across ALL sublayers, not just recurrent/SSM'
- Source: [4.1, Results 6-8]

File Paths
- Light slides: `output/slides/light/slide_12.html` through `slide_22.html`
- Dark slides: `output/slides/dark/slide_12.html` through `slide_22.html`

Done Criteria
- 22 HTML files exist (11 light + 11 dark)
- Each is self-contained, renders at 1280×720

```
- Canon formula slide has properly rendered KaTeX math
- Architecture SVG diagram has no text overlap, uses palette colors
- ≥7 slides have figures
```

INSTRUCTIONS

```
Execute the task described above. Focus on completing the task thoroughly and correctly.
- Use appropriate tools to accomplish the task
- Report progress and any issues encountered
- Verify your work when possible
- Be thorough but efficient
- When you make code changes, commit them with a descriptive commit message before finishing.
Do not leave uncommitted changes.
```

#4 Slides 23-33: Part 4.2 Scale Results, Error Analysis, Conclusion

PROJECT VISION

▶ 47,429 chars — click to expand

TASK CONTEXT

```
This task is at depth 2 in the plan graph. It has 3 upstream dependencies. 3 tasks depend on
this one. 4 other tasks are running in parallel.
```

CURRENT TASK

```
**Slides 23-33: Part 4.2 Scale Results, Error Analysis, Conclusion**

Generate HTML slides 23-33 (both light and dark themes) covering Part 4.2 real-world
scaling results, in-depth error analysis, and conclusions.

## Slides to Generate

### Slide 23 — Results 9-10: Final Controlled Comparison [4.1] (Layout 5: Three-Zone)
- Top-left: Result 9 (GLA+Canon-AbCD matches most linear models → current architecture
direction may warrant re-evaluation) + Result 10 ranking across all 5 tasks:
  - Reasoning depth: RoPE ≈ NoPE >> Linear models (4× deeper)
  - Knowledge capacity: Linear >> Transformer (1.4× capacity)
  - Reasoning breadth: GDN ≥ RoPE ≈ NoPE ≈ GLA > Mamba2
- Top-right: Use heatmaps from fig:final-compare or create SVG comparison table
- Bottom: 'With Canon, the landscape shifts drastically — GLA+Canon is surprisingly
competitive'
- Source: [4.1, Results 9-10]

### Slide 24 — Section Divider (Layout 2): 'Part 4.2: Scaling to Reality'
- Section number: 06
- Subtitle: 'Where synthetic pretraining resonates in real-world results'

### Slide 25 — Part 4.2 Experimental Setup (Layout 4: Left-Right Split)
- Left: 16 Llama/LlamaCanon models (1B, 3B, 8B) on Nemotron-CC (1T-2T tokens). 48 linear
models (GLA5, GDN2, Mamba2 ± Canon). Controlled: same data, same hyperparameters, 2-3
learning rates each.
- Right: Use `PhysicsLM4/canon_llama_results/table-params.png` showing model configurations
- Source: [4.2]

### Slide 26 — Part 4.2 Transformer Results: LlamaCanon Wins (Layout 5: Three-Zone)
- Top-left: LlamaCanon consistently surpasses Llama in ALL 8 controlled comparisons. ~2%
MMLU gain. Competitive with open-source models at same compute.
- Top-right: Use `PhysicsLM4/canon_llama_results/table-performance.png` (benchmark table)
- Bottom: 'Validates Part 4.1 Results 2+3: Canon improves Transformers at 1-8B real
pretraining scale'
- Source: [4.2]

### Slide 27 — Part 4.2 Training Curves + Compute Efficiency (Layout 4: Left-Right Split)
- Left: MMLU training curves show LlamaCanon consistently above Llama throughout training,
not just at convergence. Performance scales with model size as expected.
- Right: Use `PhysicsLM4/canon_llama_results/curve-mmlu.png` and/or `model-training-
```

time.png`
- Source: [4.2]

Slide 28 — Part 4.2 Linear Models at Scale: Key Findings (Layout 5: Three-Zone)
- Top-left: Finding 1: Canon yields significant gains for GLA. Finding 2: GLA+Canon matches or exceeds GDN and outperforms Mamba2 across 1B/3B/8B. This confirms Result 9 from Part 4.1.
- Top-right: Use `PhysicsLM4/canon_linear_results/table-performance-illustration.png` (the summary figure)
- Bottom: 'Much of GDN/Mamba2 advantage vanishes once Canon is added → suggests these architectures largely replicate Canon-like mixing'
- Source: [4.2, validates 4.1 Results 6-9]

Slide 29 — Part 4.2 Linear Training Curves (Layout 4: Left-Right Split)
- Left: MMLU curves for GLA/GDN models ± Canon at 1B/3B/8B scale. Canon provides consistent improvement throughout training.
- Right: Use `PhysicsLM4/canon_linear_results/mmlu-gla.png` and `mmlu-gdn.png` side by side
- Source: [4.2]

Slide 30 — Section Divider (Layout 2): 'Error Analysis'
- Section number: 07
- Subtitle: 'Why linear models struggle — and the limits of current architectures'

Slide 31 — Deep Dive: Memory Dynamics Bottleneck (Layout 5: Three-Zone)
- Top-left: Linear models fail NOT from insufficient memory (each Mamba2 layer has 128d parameters — hundreds of times more than needed). The bottleneck is MEMORY DYNAMICS — accumulated errors in compression and retrieval compound across hops.
- Top-right: Use training curve figures from `paper_source/canon-figs5-arxiv/curve-perm_multi_4/` showing 1-hop vs 2-hop vs 4-hop accuracy over training
- Bottom: 'Result 11 [4.1]: This is the Achilles heel of linear architectures — hybrid Transformer+Linear approaches are the practical path forward'
- Source: [4.1 Result 11 + 4.2 validation]

Slide 32 — In-Context Retrieval Failures at Scale [4.2] (Layout 5: Three-Zone)
- Top-left: ALL linear models struggle with in-context retrieval even for short contexts (~100 tokens). Even 8B models with 1T tokens fail 2-hop reasoning at ALL context lengths. This is remarkable — the simplest multi-hop task defeats all architectures.
- Top-right: Use multi-hop results columns from linear_model_best-0.pdf or create SVG bar chart showing 1-hop vs 2-hop accuracy by architecture
- Bottom: 'Explains emergence of hybrids: Qwen3-Next (Transformer+GDN), Falcon-H1 (Transformer+Mamba2)'
- Source: [4.2, validates 4.1 Result 11]

Slide 33 — Conclusion & Future Directions (Layout 3: Full Bullet)
- Canon layers: simple yet powerful — like residual connections or LoRA
- Synthetic playground: cost-effective, principled path for architecture science
- Key open questions: dynamic Canon, fine-grained Canon design, enriching synthetic tasks
- Call to action: Code at github.com/facebookresearch/PhysicsLM4, models on HuggingFace
- Generate SVG: Summary diagram showing Canon as a universal add-on across architectures
- Source: [4.1 + 4.2]

File Paths
- Light slides: `output/slides/light/slide_23.html` through `slide_33.html`
- Dark slides: `output/slides/dark/slide_23.html` through `slide_33.html`
- Copy needed images to `output/figures/` from PhysicsLM4 directories

```
## Done Criteria
- 22 HTML files exist (11 light + 11 dark)
- Part 4.2 gets ≥6 dedicated slides with detailed result analysis
- Error analysis slides include specific numbers and figure evidence
- Every result attributed to source (4.1 or 4.2)
- Multi-hop and retrieval failure analysis is in-depth with data from the result PDFs/PNGs
```

INSTRUCTIONS

```
Execute the task described above. Focus on completing the task thoroughly and correctly.
- Use appropriate tools to accomplish the task
- Report progress and any issues encountered
- Verify your work when possible
- Be thorough but efficient
- When you make code changes, commit them with a descriptive commit message before finishing.
Do not leave uncommitted changes.
```

#5 Blog Post Generation

PROJECT VISION

▶ 47,429 chars — click to expand

TASK CONTEXT

```
This task is at depth 2 in the plan graph. It has 3 upstream dependencies. 3 tasks depend on
this one. 4 other tasks are running in parallel.
```

CURRENT TASK

```
**Blog Post Generation**

Generate a detailed Markdown blog post (~3000-4000 words) covering both Part 4.1 and Part
4.2.

## Output
Write to `output/blog/post.md`

## Structure (following SKILL.md blog format)

```
Physics of Language Models: The Magic of Canon Layers
A detailed summary of Part 4.1 (NeurIPS 2025) and Part 4.2 by Zeyuan Allen-Zhu

---

TL;DR
[3 sentences: Canon layers are lightweight conv1d components that dramatically improve
reasoning across all architectures. Validated on synthetic tasks and 1-8B scale real
pretraining.]

Motivation: Why Architecture Comparison is Broken
[~400 words: 3 challenges, noise at academic scale, the physics analogy]

The Synthetic Pretraining Playground [4.1]
[~500 words: 5 tasks described, design criteria, initial comparison showing GLA weakest]

Canon Layers: The Key Innovation [4.1]
[~500 words: associative recall problem, Canon formula with LaTeX, Canon-ABCD integration,
implementation simplicity, musical analogy]

When Transformer Meets Canon [4.1]
[~400 words: Results 2-5 with specific numbers, tables for key comparisons]

When Linear Models Meet Canon [4.1]
[~400 words: Results 6-9, GLA+Canon insight]

Scaling to Reality: Part 4.2 Results
[~600 words: Detailed coverage of 4.2 — setup (16+48 models), Transformer results
(LlamaCanon consistent 2% MMLU gain), linear model results (GLA+Canon ≥ GDN), training
curves, comparison with open models]
```
```

```
## Error Analysis: Why Deep Reasoning Fails [4.1 + 4.2]
[~400 words: Memory dynamics bottleneck, retrieval errors compound, 2-hop failure at all
scales, implications for hybrid architectures]

## Key Takeaways and Future Directions
[~300 words: Canon as architectural primitive, synthetic benchmarking, open questions]

## References
[Proper citations with arXiv/SSRN links]
```

```
## Figure Requirements
- Include ALL key figures from slides (minimum 8 figures)
- Each figure with descriptive standalone caption
- Embed using `![Caption](../figures/filename.png)` or inline SVG
- Required figures: noise comparison, 5-task overview, Canon illustration, Canon
integration diagram (SVG), Transformer+Canon results, linear model results, Part 4.2
performance table, Part 4.2 training curves, error analysis training curves

## Content Requirements
- **Detailed style**: 3000-4000 words, full methodology walkthrough
- Include Canon layer equation with KaTeX-compatible LaTeX
- Include comparison tables (Markdown) for key results
- Code snippet from `canon_helper.py` showing the simplicity
- Always attribute to source (4.1 or 4.2)
- Specific numbers: 2-4× reasoning depth, 0.45% parameter increase, ~2% MMLU gain at scale
- Link to: arXiv paper, SSRN, GitHub repo, HuggingFace models

## Working Directory
`/Users/zhuoran_cisco/Documents/astro-outputs/presentation/physcics-llm-2/`

## Done Criteria
- `output/blog/post.md` exists with 3000-4000 words
- TL;DR at top
- ≥8 figures with captions
- All claims backed by specific numbers
- Proper citation format with links
- Code snippet included
- Covers both 4.1 and 4.2 in depth
```

INSTRUCTIONS

```
Execute the task described above. Focus on completing the task thoroughly and correctly.
- Use appropriate tools to accomplish the task
- Report progress and any issues encountered
- Verify your work when possible
- Be thorough but efficient
- When you make code changes, commit them with a descriptive commit message before finishing.
Do not leave uncommitted changes.
```

#6 Poster Generation

PROJECT VISION

▶ 47,429 chars — click to expand

TASK CONTEXT

```
This task is at depth 2 in the plan graph. It has 3 upstream dependencies. 3 tasks depend on
this one. 4 other tasks are running in parallel.
```

CURRENT TASK

```
**Poster Generation**

Generate a single-page portrait A0 poster (841mm × 1189mm) as HTML with CSS grid layout.

## Output
Write to `output/poster/poster.html`

## Layout (Portrait, 3-column CSS grid)
Follow SKILL.md poster format exactly.

### Header (spans all 3 columns)
- Title: 'Physics of Language Models: Part 4 — Architecture Design and the Magic of Canon
Layers'
- Author: Zeyuan Allen-Zhu (FAIR at Meta)
- Conference: NeurIPS 2025 (Part 4.1) + Code Release (Part 4.2)
- Accent line below

### Column 1 (Left): Motivation + Background + Synthetic Playground
- **Section: Motivation** (~200 words): Why architecture comparison at academic scale is
unreliable. 3 challenges. The physics analogy.
- **Section: Background** (~150 words): Architecture families overview with mini SVG
taxonomy diagram.
- **Section: Synthetic Playground**: 5 tasks with brief descriptions. Include small SVG or
figure of the task overview from plots.pdf.
- **Key Takeaway Box**: 'Synthetic tasks isolate atomic skills, enabling clean
architectural comparisons at low cost.'
- Target: 95-100% filled

### Column 2 (Middle): Canon Layers + Method + Part 4.1 Key Results
- **Section: Canon Layers** (~200 words): Formula, Canon-ABCD integration points, musical
analogy. Include Canon layer SVG diagram.
- **Section: Key Insight** (callout box): 'Canon layers add horizontal residual links —
like residual connections but across neighboring tokens. Even random-weight Canon notably
enhances performance.'
- **Section: Part 4.1 Results**: Transformer+Canon (2-4× reasoning depth), Linear+Canon
(GLA+Canon matches Mamba2/GDN). Include summary comparison figure.
- Target: 95-100% filled

### Column 3 (Right): Part 4.2 Results + Error Analysis + Conclusion
- **Section: Scaling to Reality [4.2]** (~250 words): 16 Llama + 48 linear models at 1-8B
scale. LlamaCanon consistently outperforms. GLA+Canon ≥ GDN. Include performance table
```

```
figure.
- **Section: Error Analysis** (~200 words): Memory dynamics bottleneck, 2-hop failure,
retrieval struggles. Include key figure.
- **Section: Conclusion & Future** (~100 words): Canon as universal architectural
primitive.
- **QR Code + Citation Block** at bottom
- Target: 95-100% filled

## Column Balancing
- CRITICAL: All 3 columns must be ≥95% filled
- Left column: If short, add a Background/Related Work section or larger figure
- Right column: If short, expand error analysis or add a 'Broader Impact' section
- Do NOT use flex-grow to fake balance

## Technical Requirements
- Use SKILL.md poster CSS grid layout exactly
- Body dimensions: width 841mm, height 1189mm
- Font sizes: Title 72px, Section heading 36px, Body 24px, Caption 18px
- Colors: Follow light theme palette
- Include KaTeX for Canon formula
- Embed figures as <img> with proper sizing for poster scale
- All SVGs must have larger labels (18px+ for poster viewing distance)

## Figures to Include
- Canon layer diagram (SVG or from plots.pdf page 6)
- 5-task overview (from plots.pdf)
- Key result comparison (from Part 4.1 final comparison or SVG summary)
- Part 4.2 performance illustration (table-performance-illustration.png)
- Training curves (curve-mmlu.png or mmlu-gla.png)

## Working Directory
`/Users/zhuoran_cisco/Documents/astro-outputs/presentation/physcics-llm-2/`

## Done Criteria
- `output/poster/poster.html` exists
- Renders at A0 (841mm × 1189mm) without overflow
- 3-column grid with all columns ≥95% filled
- ≥5 figures/visuals included
- QR code or URL to paper/code
- Canon formula rendered via KaTeX
- Text readable at poster viewing distance
```

INSTRUCTIONS

```
Execute the task described above. Focus on completing the task thoroughly and correctly.
- Use appropriate tools to accomplish the task
- Report progress and any issues encountered
- Verify your work when possible
- Be thorough but efficient
- When you make code changes, commit them with a descriptive commit message before finishing.
Do not leave uncommitted changes.
```

#7 Figure Extraction and Preparation

PROJECT VISION

▶ 47,429 chars — click to expand

TASK CONTEXT

```
This task is at depth 1 in the plan graph. It has 1 upstream dependencies. 12 tasks depend on
this one.
```

CURRENT TASK

```
**Figure Extraction and Preparation**

Extract and prepare all figures needed by slides, blog, and poster. This task creates the
shared `output/figures/` directory that all deliverables reference.

## What to do

### 1. Extract PDF pages as images
The main paper figures are in `paper_source/canon-figs5-arxiv/plots.pdf`. Use Python
(pypdf/pdf2image or similar) or Bash tools to extract individual pages as PNG:
- Page 3: Design criteria figure → `output/figures/design-criteria.png`
- Page 4: Five synthetic tasks overview → `output/figures/five-tasks.png`
- Page 5: Associative recall illustration → `output/figures/associative-recall.png`
- Page 6: Canon layer illustration → `output/figures/canon-illustration.png`
- Page 7: Real-life experiment results → `output/figures/real-life-results.png`
- Page 8: Noisy pretraining comparison → `output/figures/noisy-comparison.png`
- Page 9: Five tasks with diagrams → `output/figures/five-tasks-detail.png`

### 2. Copy Part 4.2 figures
Copy key images from the PhysicsLM4 repository:
- `PhysicsLM4/canon_llama_results/model-training-time.png` → `output/figures/model-
training-time.png`
- `PhysicsLM4/canon_llama_results/curve-mmlu.png` → `output/figures/curve-mmlu.png`
- `PhysicsLM4/canon_llama_results/table-performance.png` → `output/figures/table-
performance-transformer.png`
- `PhysicsLM4/canon_llama_results/table-params.png` → `output/figures/table-params-
transformer.png`
- `PhysicsLM4/canon_linear_results/table-performance-illustration.png` →
`output/figures/table-performance-linear.png`
- `PhysicsLM4/canon_linear_results/mmlu-gla.png` → `output/figures/mmlu-gla.png`
- `PhysicsLM4/canon_linear_results/mmlu-gdn.png` → `output/figures/mmlu-gdn.png`
- `PhysicsLM4/canon_linear_results/table-params-brief.png` → `output/figures/table-params-
linear.png`

### 3. Extract experiment heatmaps
From `paper_source/canon-figs5-arxiv/` subdirectories, extract page 1 of key PDF result
files:
- `perm_4/Llama_RoPE_-original.pdf` page 1 → `output/figures/depo-rope-original.png`
- `perm_4/Llama_RoPE_-Res-Canon-ABCD.pdf` page 1 (if exists) → `output/figures/depo-rope-
canon.png`
- Similar for GLA, Mamba2, GDN comparison pairs
```

- `real-life/slimp-1B-3rd-0.pdf` page 1 → `output/figures/real-life-slimp.png`
- `real-life/fwedu-1B-3rd-0.pdf` page 1 → `output/figures/real-life-fwedu.png`

4. Convert linear model result PDFs
- `PhysicsLM4/canon_linear_results/linear_model_best-0.pdf` → `output/figures/linear-model-best-0.png`
- `PhysicsLM4/canon_linear_results/linear_model_best-1.pdf` → `output/figures/linear-model-best-1.png`

Implementation
Use Python with `pdf2image` (poppler), or `sips` (macOS built-in), or Chrome headless to render PDFs to PNG. If pdf2image isn't available, try:
```bash
# macOS: use sips or qlmanage for basic conversion
# Or: python3 with PyMuPDF (fitz)
pip3 install PyMuPDF
python3 -c "import fitz; doc=fitz.open('input.pdf'); page=doc[0];
pix=page.get_pixmap(dpi=200); pix.save('output.png')"
```

Create `output/figures/` directory structure first.

Working Directory
`/Users/zhuoran_cisco/Documents/astro-outputs/presentation/physcics-llm-2/`

Done Criteria
- `output/figures/` directory exists with ≥15 PNG files
- All key paper figures extracted from plots.pdf
- All Part 4.2 figures copied
- Images are readable at slide resolution (≥1000px wide for full-width figures)

INSTRUCTIONS

Execute the task described above. Focus on completing the task thoroughly and correctly.
- Use appropriate tools to accomplish the task
- Report progress and any issues encountered
- Verify your work when possible
- Be thorough but efficient
- When you make code changes, commit them with a descriptive commit message before finishing. Do not leave uncommitted changes.

#8 PDF Assembly and Quality Check

PROJECT VISION

▶ 47,429 chars — click to expand

TASK CONTEXT

```
This task is at depth 3 in the plan graph. It has 16 upstream dependencies. 2 tasks depend on
this one.
```

CURRENT TASK

```
**PDF Assembly and Quality Check**

Combine all HTML slides into PDFs (light and dark themes), render poster to PDF, and run
quality checks across all deliverables.

## Step 1: Combine Light Theme Slides
Using the approach from SKILL.md Section 6:

1. Create `output/slides/light/combined.html` by extracting <body> and <style> from each
slide HTML file (slide_01.html through slide_33.html), wrapping each in a `.slide` div with
`page-break-after: always`.
2. Render to PDF using Chrome headless or puppeteer-core:
```bash
"/Applications/Google Chrome.app/Contents/MacOS/Google Chrome" \
  --headless --disable-gpu --no-margins \
  --print-to-pdf=./output/slides/slides-light.pdf \
  --print-to-pdf-no-header \
  "file://$(pwd)/output/slides/light/combined.html"
```

Step 2: Combine Dark Theme Slides
Same process for dark theme:
1. Create `output/slides/dark/combined.html`
2. Render to `output/slides/slides-dark.pdf`

Step 3: Render Poster to PDF
Use puppeteer-core or Chrome headless with custom A0 dimensions:
```bash
npm install puppeteer-core
node -e "
const puppeteer = require('puppeteer-core');
(async () => {
  const browser = await puppeteer.launch({ executablePath: '/Applications/Google
Chrome.app/Contents/MacOS/Google Chrome', headless: 'new' });
  const page = await browser.newPage();
  await page.goto('file://' + require('path').resolve('./output/poster/poster.html'), {
waitUntil: 'networkidle0' });
  await page.pdf({ path: './output/poster/poster.pdf', width: '841mm', height: '1189mm',
printBackground: true, margin: { top: 0, right: 0, bottom: 0, left: 0 } });
  await browser.close();
})();
```
```

"
```

## Step 4: Quality Checklist
Verify all deliverables against SKILL.md quality checklist:

### Slides
- [ ] One idea per slide — no information overload
- [ ] Max 6-8 bullet points per Full Bullet slide
- [ ] Every equation has plain-English annotation
- [ ] At least 60% of slides have a figure (target: ≥20 of 33 slides)
- [ ] All original paper figures referenced in outline are included
- [ ] New SVG figures use palette colors with no text overlap
- [ ] Max 2-3 accent colors per slide
- [ ] Title slide has paper title, authors, affiliation
- [ ] Section dividers introduce each major topic
- [ ] KaTeX renders correctly in all math slides
- [ ] PDF renders clean without clipping or overflow
- [ ] Both light and dark PDFs generated

### Blog Post
- [ ] TL;DR at top
- [ ] Word count 3000-4000
- [ ] ≥8 figures with descriptive captions
- [ ] All claims backed by specific numbers
- [ ] Proper citations with links
- [ ] Both 4.1 and 4.2 covered in depth

### Poster
- [ ] Fits single A0 page without overflow
- [ ] 3-column grid, all columns ≥95% filled
- [ ] Text readable at poster distance (24px+ body)
- [ ] Key figures prominent and legible
- [ ] QR code or URL included

## Step 5: Cross-Check Consistency
- Verify key numbers match across all 3 deliverables
- Ensure all result attributions (4.1 vs 4.2) are consistent
- Check that Canon formula is identical in slides, blog, and poster

## Working Directory
`/Users/zhuoran_cisco/Documents/astro-outputs/presentation/physcics-llm-2/`

## Output Files
- `output/slides/slides-light.pdf`
- `output/slides/slides-dark.pdf`
- `output/poster/poster.pdf`
- `output/quality-report.md` (checklist results)

## Done Criteria
- Both slide PDFs exist and are multi-page (33 pages each)
- Poster PDF exists at A0 dimensions
- Quality report confirms all checklist items pass
- No rendering artifacts (clipping, overflow, missing figures)
```

INSTRUCTIONS

```
Execute the task described above. Focus on completing the task thoroughly and correctly.
- Use appropriate tools to accomplish the task
- Report progress and any issues encountered
- Verify your work when possible
- Be thorough but efficient
- When you make code changes, commit them with a descriptive commit message before finishing.
Do not leave uncommitted changes.
```

#9 Content Review and Narrative Polish

PROJECT VISION

▶ 47,429 chars — click to expand

TASK CONTEXT

```
This task is at depth 4 in the plan graph. It has 17 upstream dependencies. 1 tasks depend on
this one.
```

CURRENT TASK

```
**Content Review and Narrative Polish**

Human review of all deliverables for content quality, narrative flow, and technical
accuracy.

## Review Criteria

### Narrative Flow (Slides)
1. Does the presentation tell a compelling story from problem → insight → evidence →
implications?
2. Are transitions between slides logical? Does each slide connect to the next?
3. Is the audience engagement maintained? (Expert in mechanistic interpretability or LM
training)
4. Is Part 4.2 given sufficient depth? (≥6 slides + in-depth error analysis)
5. Are all results properly attributed to 4.1 or 4.2?

### Technical Accuracy
1. Is the Canon layer formula correct: $h'_t = w_0 \odot h_t + w_1 \odot h_{t-1} + w_2
\odot h_{t-2} + w_3 \odot h_{t-3}$?
2. Are the 12 results accurately summarized?
3. Are the Part 4.2 benchmark numbers consistent with the source tables?
4. Is the error analysis (memory dynamics, not memory size) accurately explained?
5. Are architecture descriptions (GLA, Mamba2, GDN) technically precise?

### Visual Quality
1. Do all SVG diagrams follow the palette and have no text overlap?
2. Are original paper figures readable at slide/poster scale?
3. Is the figure-to-slide ratio ≥60% for content slides?
4. Do both light and dark themes look polished?

### Blog Post
1. Is the writing clear and engaging for the target audience?
2. Are equations explained in plain English?
3. Do figure captions stand alone (readable without surrounding text)?
4. Is the code snippet from canon_helper.py illuminating?

### Poster
1. Are all 3 columns balanced (≥95% filled)?
2. Is the visual hierarchy clear (title → sections → body)?
3. Can the poster be read in 3-5 minutes?
```

```
## Output
`output/review-notes.md` with any corrections needed.

## Done Criteria
- All deliverables reviewed
- Any critical errors flagged and corrected
- Narrative flow confirmed as compelling
```

INSTRUCTIONS

```
Execute the task described above. Focus on completing the task thoroughly and correctly.
- Use appropriate tools to accomplish the task
- Report progress and any issues encountered
- Verify your work when possible
- Be thorough but efficient
- When you make code changes, commit them with a descriptive commit message before finishing.
Do not leave uncommitted changes.
```

#10 All Deliverables Complete

PROJECT VISION

▶ 47,429 chars — click to expand

TASK CONTEXT

```
This task is at depth 5 in the plan graph. It has 18 upstream dependencies.
```

CURRENT TASK

```
**All Deliverables Complete**

All three deliverables (slides in both themes, blog post, poster) have been generated,
rendered to PDF, and quality-checked. Final outputs:
- `output/slides/slides-light.pdf` (~33 slides, light theme)
- `output/slides/slides-dark.pdf` (~33 slides, dark theme)
- `output/blog/post.md` (detailed blog post, 3000-4000 words)
- `output/poster/poster.pdf` (A0 portrait poster)
- `output/quality-report.md` (quality checklist results)
```

INSTRUCTIONS

```
Execute the task described above. Focus on completing the task thoroughly and correctly.
- Use appropriate tools to accomplish the task
- Report progress and any issues encountered
- Verify your work when possible
- Be thorough but efficient
- When you make code changes, commit them with a descriptive commit message before finishing.
Do not leave uncommitted changes.
```